Discussion

What the demonstration does and does not show I

The demonstration shows that the registered analysis, run against seeded data with a genuine effect (`Normal(0.8, 1)` treatment versus `Normal(0, 1)` control), recovers a significant confirmatory result: an observed mean difference of `1.003` with a two-sided permutation p-value of `0.0005`. It does **not** show anything about any real-world phenomenon. The data are synthetic and the effect is injected by construction; the number exists to prove that the locked plan, when executed, produces an auditable statistic — not to support a scientific claim. This is the honest reading of a template: the value is a property of the code path, not evidence about the world.

Why the boundaries are enforced in code I

Prose conventions for "confirmatory versus exploratory" are easy to blur once results are in view. Here the boundary is a function contract: `compare_analysis_to_registration` refuses unregistered outcomes without a documented deviation, `build_deviation_ledger` assigns a severity to every executed element, and `build_review_packet` partitions confirmatory from exploratory outcomes deterministically. The content hash on the frozen registration makes silent edits to the locked plan detectable (`hash_mismatch`). Together these turn registered-report discipline into checks that fail loudly rather than guidance that can be quietly ignored.

Limitations and scope I

- The demonstration uses a single hypothesis and a single confirmatory outcome; real registrations will declare several, and the same helpers scale to them.
- A permutation test is used because it is exact, deterministic, and dependency-free; forks may register any primary model by naming it in the plan.
- Ethics-review and registered-report-stage metadata in the fixture are synthetic (`status: exempt`, `authority: synthetic-fixture-review`) and must be replaced with real values before any submission.

Using this template I

Fork with `scripts/audit/copy_exemplar.py`, replace `data/example_registration.json` with your own preregistration, swap the synthetic generator in `src/registered_report/demo_study.py` for real data collection, record stage and ethics metadata, and rerun the tests and figure generation. Keep confirmatory claims tied to registered outcomes, record every departure in the deviation ledger, and never let post-run result prose rewrite the preregistered intent.